Phase 1: Memory-Aware Agent Edge Serving

CSE 291A, Fall 2026, Option 1. November 4, 2026.

Figures and full tables are in the appendix. The numbers below are means across repeats unless noted otherwise.

1 What we wanted to know

We wanted to know how a FIFO scheduler performs on one laptop with a fixed KV cache budget, and which problems dominate its performance. Our setup uses a multi-step agent that calls the model, waits on a tool, and then calls the model again with a longer prompt. While the tool is running the model is free, but the KV cache from the previous step still retains useful information, and on a 16 GB laptop that cache is the limited resource. A fixed-concurrency FIFO scheduler can fail here for two main reasons. The first reason is that too few agents are allowed in at once, resulting in idle time. For example, if only one agent is in the system, it finishes a model step and then waits on a tool, and during the wait the model has no other request to run. The model then sits idle when it could be processing other tasks. The second reason is admitting too many agents at once. For example, agent A has already run, so its slot holds the tokens from that call. Agent B then gets the same slot and overwrites them. When A comes back, the new prompt still starts with those same tokens, but the server has to recompute them all over again before it can read the new tool result. We implemented that FIFO in front of one `llama-server`, on a real agent benchmark and on controlled prompts, to see which of those two effects dominates and what a later policy would have to beat. We do not implement a new policy in this phase.

2 Implementation

We ran the experiments on a MacBook Air M4 with 16 GB of unified memory, running macOS 15.3, and the runs were taken on AC power. The model was Qwen3-4B at Q4_K_M with thinking disabled, the KV cache had f16 precision, and a token took up roughly 144 KiB. The server was llama.cpp `llama-server` build 11669. We fixed the build since slot save, restore, and the cache flags can vary across builds. We later repeated the full matrix on an AWS T4 GPU with the same model, build and workloads, and measured run-to-run noise on a second Mac (M5); those numbers are never mixed with the M4 numbers. The server launcher starts a single server process with P slots and a total context budget, typically 24,576 tokens, which is split across the slots. The server allocates the entire KV pool at startup, so its memory usage stays roughly the same when the queue experiences more traffic.

The queue sits in front of that server. It takes steps that are already ready and runs up to K of them in that ready order; it does not keep the order in which the tasks first showed up. A tool wait happens outside the queue, so a task that is blocked on a tool does not hold one of the K permits. An optional cap J restricts how many agent tasks can be in the system at once, and time spent waiting for J counts toward latency. K only limits model calls that are ready to run, while J limits how many whole agents are allowed in.

The live capture is the only place where a real agent runs. We had the agent execute τ^2-bench Retail v1.0.1 with the official tools, orchestrator, and environment checker, with local Qwen as the agent and DeepSeek as the user simulator. A small adapter cleans a tool call's JSON: it turns a plain-text tool call into a `tool_calls` entry that the Retail benchmark recognizes, but only when the text is a JSON object with a name and an arguments object and the name is an existing Retail tool. Otherwise it leaves the message alone.

3 Workload

We sampled 20 eval tasks and 10 dev tasks with seed 291 and kept every task, even the ones that failed. Eval tasks take roughly 2 to 20 model calls, with a median of 11, and the longest prompt in a task is between 5,030 and 9,677 tokens. The environment check passed on only 3 of the 20 eval tasks. The 4B model is weak at this Retail task, but we did not drop these failures. The failed tasks do not loop: the passing tasks (12 to 16 calls) lie inside the range of the failing ones, and six failures end within 4 calls (Table A7), so a stronger agent would produce longer histories. We used the dev tasks only to pick a baseline.

We ran each Retail task once and saved its shape: how many model calls there were, how long each answer was, and how much the prompt grew after each tool result. The 20 eval recordings and the 10 dev recordings are frozen as the workload.

The timed runs did not call DeepSeek and did not run the shop again. Instead, they played the recording back. Suppose a task has already called the model once. The next prompt is that same prompt, plus the tokens the model just wrote in this replay, plus a piece of the saved prompt. We sized that piece in tokens so the prompt grows by the same number of tokens it grew in the live run. The answer length is fixed to the live run too, so one setup cannot look faster by writing a shorter answer. We append the new tokens to the end instead of rewriting the prompt, because the server can reuse the old tokens only if the next request starts with those exact same tokens.

The benchmark runner takes 8 of those saved tasks, which arrive at random times (Poisson, on average about once per minute), and after each model step it simulates a tool with a log-normal delay (median 2 seconds). It then inserts the next step into the FIFO queue. The sweep script restarts the server when the number of slots or the cache flags change, since those are fixed at startup. Repeat 0 is the same 8 tasks, the same arrival times, and the same tool delays for every configuration. Repeat 1 is a different seeded draw of tasks, arrival times, and tool delays. If two configurations differ on repeat 0, that can be attributed to the configuration. If repeat 0 and repeat 1 differ, that can be attributed to a different mix of tasks. Finishing a task means every saved step ran. It does not necessarily mean the customer was actually helped.

We also ran two checks that are not the Retail replay. The micro benchmark used exact lengths: 1K, 4K, and 8K tokens in and 64 or 512 tokens out, with the prompt cache disabled. The cache check measured the continuation after a slot is kept, idle, overwritten, erased, or saved to SSD and restored.

4 Findings

Rebuilding a long prompt is the slow part. At $K = 1$, the prefill of 8,192 tokens took 43.6 seconds, about 184 tokens per second. Generating 512 tokens on a 1,024-token prompt took about 23.6 seconds end to end, and only 4.4 seconds of that was prefill. Losing an 8K history therefore costs more than writing the answer: its 43.6 seconds of prefill is as long as generating about 1,160 output tokens (Figure A1).

If the slot is still there, that cost almost goes away. After an 8K prompt, a continuation's prefill took about 0.17 to 0.20 seconds and reused 99.9% of the input when the slot was retained, idle, or loaded from disk. Overwrite and erase dropped to 0% reuse and took about 44 seconds. The 8K state took up 1,156 MiB; saving it took only 0.37 seconds and restoring it 0.35 seconds, and these steps combined with prefill took about 0.90 seconds. After save, overwrite, and restore, 64 greedy tokens matched the run that was never interrupted. Saving a slot to disk is cheap on this machine, but recomputing the prefix is not (Table A3).

The process does not get bigger when we add load. llama-server allocates the KV pool up front, so a 24K budget is 3.38 GiB of KV on paper; the process peaks around 3.6 GiB RSS, with about 1.9 GiB still free. Four slots at the same budget use about the same RSS. System available memory, which includes the Metal allocations that RSS misses, stayed between 1.9 and 3.5 GiB without swap-out. The memory limit shows up as agents overwriting each other's slots, or as a prompt that does not fit in its slot. It does not show up as RSS climbing with load. With two slots, a returning task finds its whole history 100% of the time when it is alone, 83% with two active tasks, 11% with three, and never with four (Table A4, Figure A3).

On the M4 retail replays, capping active tasks is the best simple setup, and it is unfair. Table 1 compares the main setups.

The 4-slot latencies look short because the long tasks are the ones that die. They fail with "context size has been exceeded." Increasing the context budget to 36K tokens, about 9K per slot, decreased the failures to 3 out of 16 tasks, and that run still does not beat the 2-slot configuration with our task cap. All 25 failed

Table 1: M4 retail replays under the default load (24K-token budget, log-normal tool delay with median 2 s, one Poisson arrival per minute). P50 and P95 average the per-run percentiles over finished tasks; ± is the SD across repeats, mostly from the task mix. Full table: Table A1.

Setup	Finished	Tasks/min	P50	P95	History prefilled again	Full history hit
2 slots, $K = 2$, no task cap (FIFO)	24/24	0.59 ± 0.20	318 s	479 s	45%	58%
2 slots, $K = 2$, at most 2 tasks ($J = 2$)	24/24	0.74 ± 0.23	152 s	256 s	20%	96%
2 slots, $K = 2$, no task cap, plus a 3 GiB host prompt cache	24/24	0.53 ± 0.11	363 s	532 s	64%	45%
4 slots, $K = 4$, 6K tokens each	10/24	0.42 ± 0.21	38 s	68 s	16%	88%

tasks are such overflows, and they stay in every denominator; no request reached the 600-second deadline.

The variation across repeats is mostly which tasks were drawn. Each repeat is a different draw of 8 tasks, and every configuration sees the same 8. One repeat is a long batch: with no cap, the median task took 747 seconds. Another repeat is a short batch, with a median of 50 seconds. On the three repeats the cap is 1.03 times, 0.97 times, and 1.97 times the uncapped FIFO, so it is a big win on one batch and about a tie on the other two. The big win is the batch where tasks overlap the most, 4.4 tasks active under FIFO against 2.4 and 0.7, not only the batch with longer prompts (Table A6).

On the long batch, the cap wins by keeping histories in their slots. With no cap, a task on that batch waits about 292 seconds in the FIFO line, spends about 255 seconds in the model round trip, and finishes after 580 seconds. Most of that model time recomputes old tokens after another agent took the slot, and longer steps lengthen the line. At $K = 2$ the round trip also includes time spent sharing the server. With the cap, the line wait is 0, a task waits about 120 seconds to be let in, the round trip falls to about 91 seconds, and the task finishes after 244 seconds. The tool wait stays about 33 seconds either way. On the M4 dev tasks, the 10 we held out, $P = 2$, $K = 2$, $J = 2$ was the best of the four runs (one run each), about 0.91 tasks per minute, and none of the prefill was old tokens being computed again. Uncapped FIFO on those tasks was about 0.45 tasks per minute, and 77% of the prefill was old tokens. On the M4, Phase 2 therefore has to beat this setup, not only uncapped FIFO.

The same cap does not win on a faster GPU. We also ran the full matrix on an AWS T4, three trials of every run, with arrivals raised to 3.5 tasks per minute so that tasks overlap about as much as on the M4. Histories are lost the same way: with two slots, a returning task finds its whole history 98% of the time alone, 37% with three active tasks, and 15% with four. But on the T4 an 8K re-prefill takes 5.1 seconds instead of 44, and a model step (1.9 seconds) is shorter than a tool wait (2.9 seconds), so the cap mostly leaves the GPU idle while admitted agents wait on tools. The $J = 2$ cap finishes tasks at 0.91, 1.00, and 0.79 times FIFO on the three repeats, and the best fixed cap on the T4 dev tasks is $J = 4$ (1.04 times FIFO). Whether a cap helps depends on how expensive a lost history is compared with a tool wait (Tables A2 and A6, Figure A4).

The cap is also unfair to short tasks. Slowdown is

$$\text{slowdown} = \frac{T_{\text{completion}}}{T_{K=1,J=1} + T_{\text{tool}}} \tag{1}$$

$T_{\text{completion}}$ is how long the task took in that run, $T_{K=1,J=1}$ is its model time when it runs alone, and T_{tool} is its own tool wait. Under the cap, short tasks, split at the median output length, slow down by 9.5 times and long tasks by 1.6 times, because the short ones are waiting to be admitted. With no cap the slowdowns are closer, about 4.5 times for short tasks and 3.4 times for long tasks, and the whole run is slower. The T4 shows the same pattern: short-task slowdown rises from 1.5 under FIFO to 8.5 under $J = 2$. A looser cap, $J = 3$, keeps short-task slowdown at 2.2 while finishing tasks at 0.98 times FIFO (Figure A6).

llama.cpp does not keep a slot for each agent. Every Retail task starts with the same 4,891-token system prompt and tool schema. llama.cpp picks the idle slot whose cached prompt shares the longest prefix, and when $K = 1$ every slot is idle, so it keeps choosing the same slot and the second slot stays empty. Two active tasks then overwrite each other in that one slot. When a history is lost, the shared prefix usually survives and only the task-specific tail is recomputed. A slot is therefore not reserved for an agent unless we pass `id_slot` ourselves.

The host prompt cache did not help. `--cache-ram 3072` is the 3 GiB row in Table 1. On the same eval mixes it was no better than no cache (0.81, 1.00, and 0.99 times FIFO on the three repeats), and on the T4 it matched FIFO exactly.

Tool delay and arrival rate change how often agents overwrite each other's old tokens. With 2 slots and $K = 2$, we first changed only the wait after each model step: no wait, the usual median 2-second wait, and a fixed 15-second wait. Each repeat is a different draw of 8 tasks, so we compared only the repeats where all three waits ran. No wait and the 2-second wait are close, 0.60 and 0.65 tasks per minute respectively, inside the long-batch versus short-batch gap. The 15-second wait is worse: 66% of the prefill is old tokens computed again, and finished tasks fall to 0.46 per minute. We then changed only how often a new task shows up. At one task every 2 minutes, a task takes 1.3 times as long as running alone, and 92% of next steps still have the whole previous prompt. At 2 tasks per minute, finished tasks increase to 0.80 per minute, P95 increases to 416 seconds, and only 55% of next steps still have the whole previous prompt. The one-per-minute value is 0.65 here, not 0.59, because only shared repeats count (Figure A7).

5 What we plan for Phase 2

For Phase 2 we want one policy whose decisions come from costs it measures while running, because the best fixed cap is $J = 2$ on the M4 and $J = 4$ on the T4. It keeps the queue and cache paths of our design memo but replaces its memory watermark, since resident memory never moves and stopping admission is the fixed cap that loses on the T4. Appendix C fills the memo's retention score with the costs measured here. First, pin each agent to a slot, so $K = 1$ stops leaving a slot unused. Second, run a ready continuation before a new task, with an aging term so a long task cannot wait forever, admit any task that has waited longer than a fixed bound, and otherwise admit a new task when the server would otherwise sit idle and the budget can hold the context growth we have actually seen. This is meant to keep the hit rate of the cap while cutting the wait that short tasks pay. Third, when a tool runs long, choose between keeping the slot, saving it to SSD, and dropping it from the measured costs. On the M4 that save is under a second, and an 8K re-prefill is 44 seconds. On the T4 the same save is 8.95 seconds against a 5.1-second re-prefill, so a save pays only when it is written during a long tool wait. We plan to log the decision time and count it in the latency measurement.

We also plan to keep the comparisons on one model, one precision, and the same output lengths. The baselines are a plain FIFO, the best fixed setup checked on dev for each machine ($J = 2$ on the M4 and $J = 4$ on the T4), `--cache-ram` as the simple fixed cache, and a fixed-TTL policy that pins a waiting agent's slot for a set time, together with queue only, cache only, and both together. We will draw more repeats from the task pool and rerun the 0.5 per minute arrival and the zero-delay case, where there is little eviction to remove, so the policy should not help much there. A speedup has to show up as less repeated prefill. Dropping tasks or shortening outputs does not count.

6 Limits

Eight tasks per run drawn from a pool of 20, and three repeats, or two for the 36K run and for the delay and arrival sweeps, mean that P95 only describes these runs and per-repeat gains depend on the task mix; the eviction curve, pooled over hundreds of continuations, is more stable. The M4 runs were done once per repeat. On the M5, three trials of repeat 0 on identical inputs varied by 11% for uncapped FIFO and 2% for the $J = 2$ cap, and the cap was ahead in every trial, so M4 differences under about 20% should be read as ties. On the T4, run-to-run variation was at most 1.3% (Table A5). Runs exclude the 0.66-second model load.

A Artifacts and reproduction

All code, workloads, raw logs and generated reports are in the project repository.

Artifact	Location
FIFO queue, request logging, append-only replay	`phase1/core.py`
Server launcher (P, budget, unified KV, host cache)	`phase1/server.py`
Live τ^2-bench capture; tool-call normalization	`phase1/capture.py`, `phase1/adapters.py`
Micro, cache and macro suites; admission cap J	`phase1/bench.py`
Summaries, noise, figures and generated reports	`phase1/summarize.py`, `phase1/noise.py`, `phase1/final.py`
Full pipeline (resumable)	`scripts/overnight.sh`
Frozen workloads (20 eval, 10 dev)	`runs/final/workload_eval.json`, `runs/final/workload_dev.json`
M4 results, generated report and figures	`runs/final/`, `runs/final-dev/`
T4 results, generated report and figures	`runs/aws-gpu/`, `runs/aws-gpu-dev/`, `REPORT.md`
T4 arrival-rate calibration pilot	`runs/aws-gpu-pilot/`
M5 run-to-run noise trials	`runs/m5-noise/`

Every number in this report is regenerated from the raw logs, without the model or a GPU, by `python -m phase1.final -results-dir runs/final -main-arrival poisson:1` (M4) and `-results-dir runs/aws-gpu -main-arrival scaled:3.5` (T4). Each run records its git commit, server command, power state and (on the T4) GPU identity in `raw/runs_meta.jsonl`. One M4 launch ran on battery; it was moved to `raw/interrupted/` and rerun on AC power.

B Full tables

Table A1: M4 baselines under the main condition (24K tokens, log-normal tool delay, 1 task/min). Tasks/min is mean ± SD across repeats, which mostly reflects the task mix of each repeat; other columns are means of per-run values. Latency covers finished tasks only.

Configuration	Repeats	Finished	Tasks/min	P50 (s)	P95 (s)	Re-prefill	Full-history hit	Mean slowdown
$P=1$, $K=1$	3	24/24	0.483 ± 0.116	382	575	78.0%	26.4%	4.61
$P=2$, $K=1$, $J=1$	3	24/24	0.677 ± 0.197	174	280	0.0%	100.0%	7.49
$P=2$, $K=1$, $J=\infty$	3	24/24	0.450 ± 0.051	412	638	78.0%	25.1%	5.15
$P=2$, $K=2$, $J=2$	3	24/24	0.736 ± 0.228	152	256	20.0%	95.7%	5.53
$P=2$, $K=2$, $J=\infty$ (FIFO)	3	24/24	0.587 ± 0.203	318	479	45.4%	57.8%	3.98
FIFO + 3 GiB host cache	3	24/24	0.532 ± 0.113	363	532	63.6%	45.0%	4.70
$P=4$, $K=4$, 24K	3	10/24	0.424 ± 0.205	38	68	15.6%	87.5%	4.23
$P=4$, $K=4$, 36K	2	13/16	0.528 ± 0.141	50	169	7.1%	97.8%	1.52
$P=4$ unified, $K=4$, 24K	3	16/24	0.465 ± 0.052	111	203	31.5%	77.3%	4.42

Table A2: T4 baselines under the main condition (24K tokens, log-normal tool delay, 3.5 tasks/min). Each row is 3 repeats × 3 trials × 8 tasks; tasks/min is mean ± SD across repeats of the per-repeat trial means.

Configuration	Finished	Tasks/min	P50 (s)	P95 (s)	Full-history hit	Re-prefill
$P=2$, $K=2$, $J=\infty$ (FIFO)	72/72	2.133 ± 0.662	65	132	57%	62%
$P=2$, $K=2$, $J=4$	72/72	2.086 ± 0.620	64	129	62%	60%
$P=2$, $K=2$, $J=3$	72/72	2.096 ± 0.627	59	128	69%	60%
$P=2$, $K=2$, $J=2$	72/72	1.875 ± 0.455	73	138	91%	39%
$P=2$, $K=1$, $J=\infty$	72/72	1.914 ± 0.546	85	147	25%	79%
$P=2$, $K=1$, $J=1$	72/72	1.229 ± 0.086	155	241	100%	0%
$P=1$, $K=1$	72/72	1.915 ± 0.547	84	147	25%	79%
FIFO + 3 GiB host cache	72/72	2.135 ± 0.663	66	132	57%	64%
$P=4$, $K=4$, 24K	30/72	1.246 ± 0.459	16	19	82%	36%
$P=4$ unified, $K=4$, 24K	48/72	1.511 ± 0.430	36	92	79%	43%
$P=4$, $K=4$, 36K (R0–R1)	39/48	1.507 ± 0.429	29	102	90%	47%

Table A3: Platform costs. Micro and cache-probe results at $K=1$.

	M4	T4
Prefill rate (tokens/s)	184	1,565
Decode rate (tokens/s)	27	66
Re-prefill of 8K history (s)	44.1	5.1
Continuation with slot kept (s)	0.20	0.03
Save 8K slot state (s)	0.37	8.95
Restore 8K slot state (s)	0.35	0.47
Model step / tool time per step (s)	9.5 / 2.9	1.9 / 2.9
Server busy under FIFO	71%	60%
Model load, median (s)	0.66	1.32

Table A4: Full-history hit rate by number of active tasks at dispatch, pooled over all runs of each server.

Active tasks	1	2	3	4	5
M4, $P=2$, 24K	100%	83%	11%	0%	0%
M4, $P=4$, 36K	100%	96%	92%	–	–
T4, $P=2$, 24K	98%	81%	37%	15%	1%
T4, $P=4$, 24K	97%	81%	72%	69%	40%
T4, $P=4$, 36K	96%	94%	81%	77%	–

Table A5: Run-to-run noise on the M5: R0 replayed three times on identical inputs.

	Trial 1	Trial 2	Trial 3	CV
FIFO, tasks/min	0.662	0.567	0.542	11%
$J=2$, tasks/min	0.739	0.732	0.764	2%

C Cost model

A continuation whose history was lost recomputes its task-specific history of H_i tokens (the shared 4,891-token prefix usually survives), so

$$C_i^{\text{reprefill}} = H_i / r_{\text{prefill}}.$$

Under contention, a miss recomputes a median of 797 tokens at 6.0 s per 1,000 tokens on the M4 (4.8 s) and 1,123 tokens at 0.7 s per 1,000 tokens on the T4 (0.8 s). The mean tool time per step is 2.9 s on both platforms.

Admission. Admitting a task beyond the slot count fills idle server time but makes returning tasks pay $C^{\text{reprefill}}$. The ratio $\rho = C^{\text{reprefill}}/T_{\text{tool}}$ compares the two: $\rho \approx 1.6$ on the M4, where the cap $J=2$ wins, and $\rho \approx 0.27$ on the T4, where it loses. Two platforms give two points and do not fix the crossover; the tool-delay sweep changes T_{tool} on one platform and is how Phase 2 will locate it.

Retention. Our design memo scores keeping agent i's state for time τ as

$$R_i(\tau) = P(D_i \le \tau)\, C_i^{\text{reprefill}} - C_i^{\text{memory}}(\tau),$$

Table A6: Completed tasks per minute relative to FIFO ($P = 2$, $K = 2$, $J = \infty$) on the same tasks, per repeat. T4 values are means of three trials. Throughput counts finished tasks only, so the 4-slot rows also lose tasks to context overflow.

	M4, 1 task/min, one run per repeat				T4, 3.5 tasks/min, three trials per repeat			
Configuration	R0	R1	R2	Finished	R0	R1	R2	Finished
FIFO, absolute tasks/min	0.82	0.49	0.45	24/24	2.47	1.37	2.56	72/72
$P = 2$, $K = 2$, $J = 4$	–	–	–	–	1.00	1.00	0.94	72/72
$P = 2$, $K = 2$, $J = 3$	–	–	–	–	1.00	1.00	0.96	72/72
$P = 2$, $K = 2$, $J = 2$	1.03	0.97	**1.97**	24/24	0.91	1.00	**0.79**	72/72
$P = 2$, $K = 1$, $J = 1$	1.01	0.93	1.65	24/24	0.53	0.83	0.48	72/72
$P = 2$, $K = 1$, $J = \infty$	0.62	0.89	0.90	24/24	0.91	0.94	0.86	72/72
FIFO + 3 GiB host prompt cache	0.81	1.00	0.99	24/24	1.00	1.00	1.00	72/72
$P = 4$, $K = 4$, 6K tokens per slot	0.42	0.55	1.45	10/24	0.44	0.64	0.69	30/72

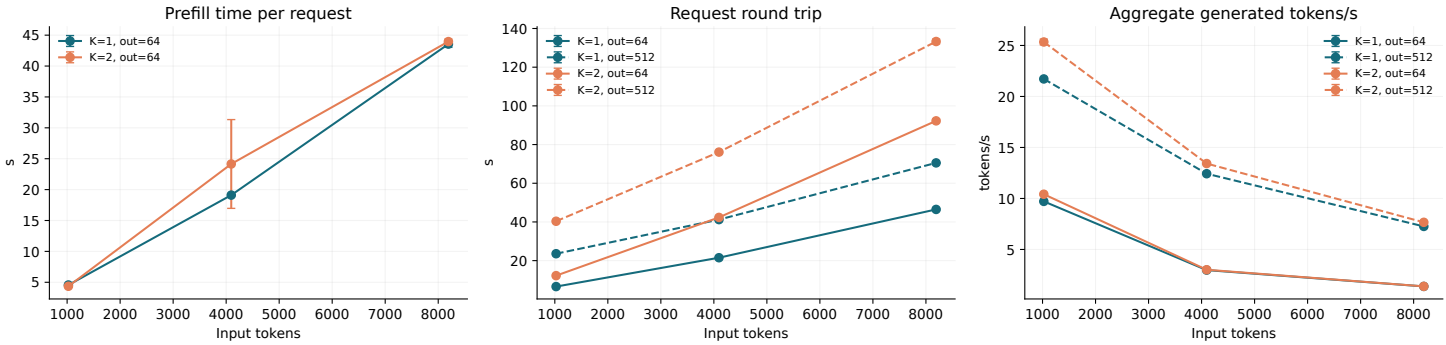
where D_i is the remaining tool time. Phase 1 supplies $C_i^{\text{reprefill}}$ from the measured prefill rate and $P(D_i \leq \tau)$ from the tool-delay distribution. Saving and releasing the slot instead of keeping it pays when the save finishes within the tool wait ($T_{\text{save}} < D_i$) and the restore is cheaper than recomputation ($T_{\text{restore}} < C_i^{\text{reprefill}}$). For an 8K state, the M4 needs 0.37 s to save and 0.35 s to restore against 44 s to recompute, so saving almost always pays; the T4 needs 8.95 s to save, so saving pays only during tool waits longer than about 9 s.

Table A7: Shape of the 20 recorded eval tasks by environment-check outcome (min / median / max).

Outcome	Tasks	Model calls	Longest prompt (tokens)	Output tokens per task (median)
Passed	3	12 / 15 / 16	6,693 / 9,302 / 9,340	833
Failed	17	2 / 8 / 20	5,030 / 6,240 / 9,677	696

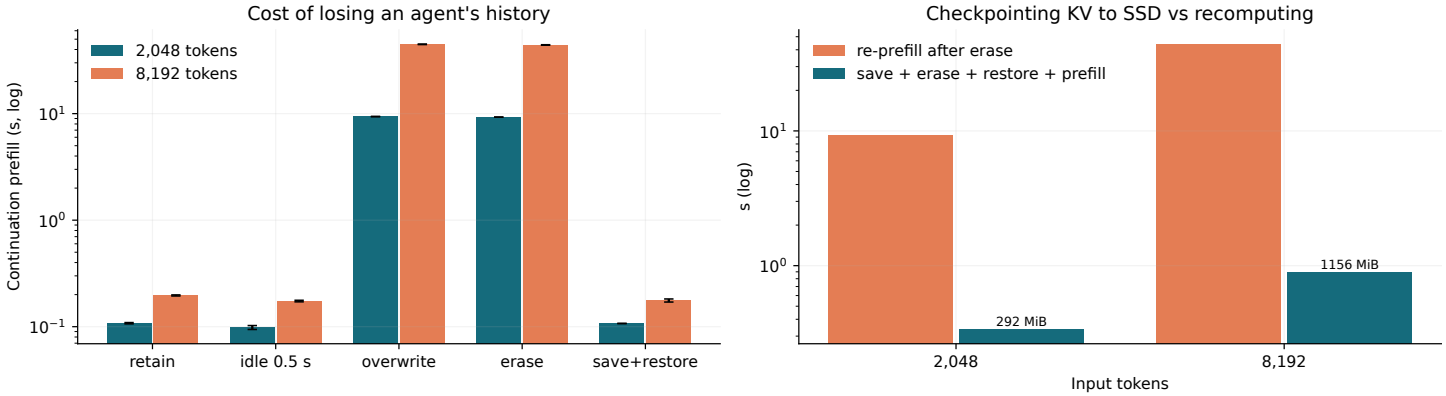
D Figures

F2 Micro: cost of context length, output length and concurrency (prompt cache off)



Mean ± SD over repeats. K requests submitted together on P=2 slots.

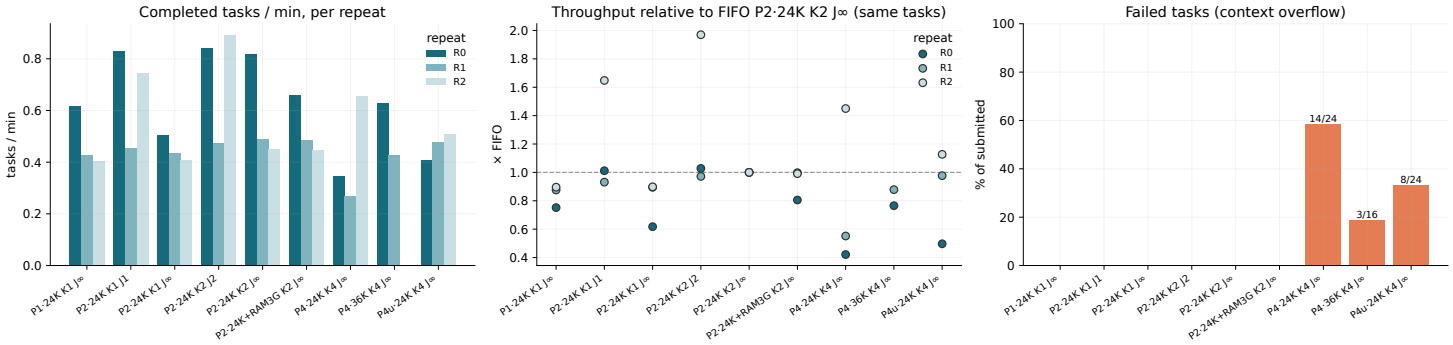
F3 Cache mechanisms exposed by llama-server (slot reuse, erase, save/restore)



Warm page cache; save is not fsync-forced. Overwrite = another request on the same slot.

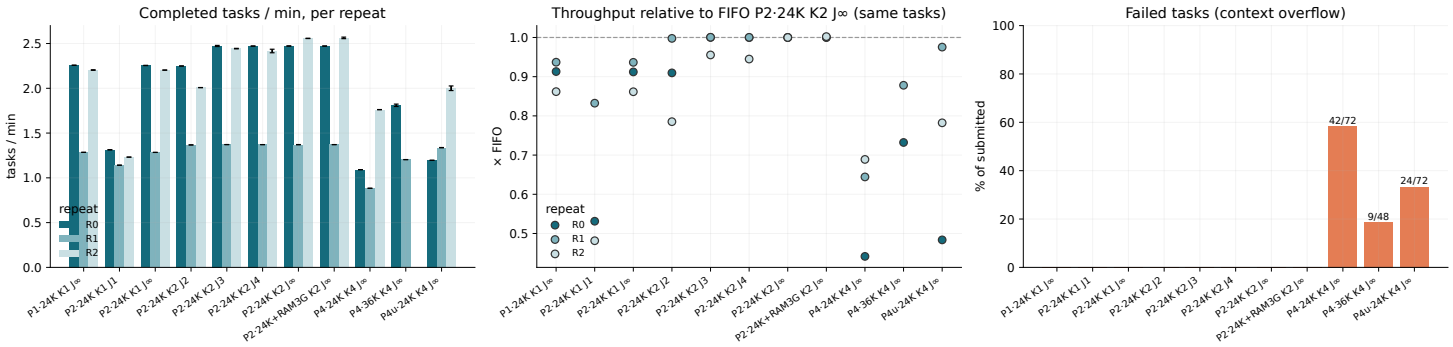
Figure A1: M4 micro benchmark and cache probe: prefill dominates long contexts, and a kept, idle or restored slot removes it.

F4 End-to-end FIFO baselines by server configuration and KV budget



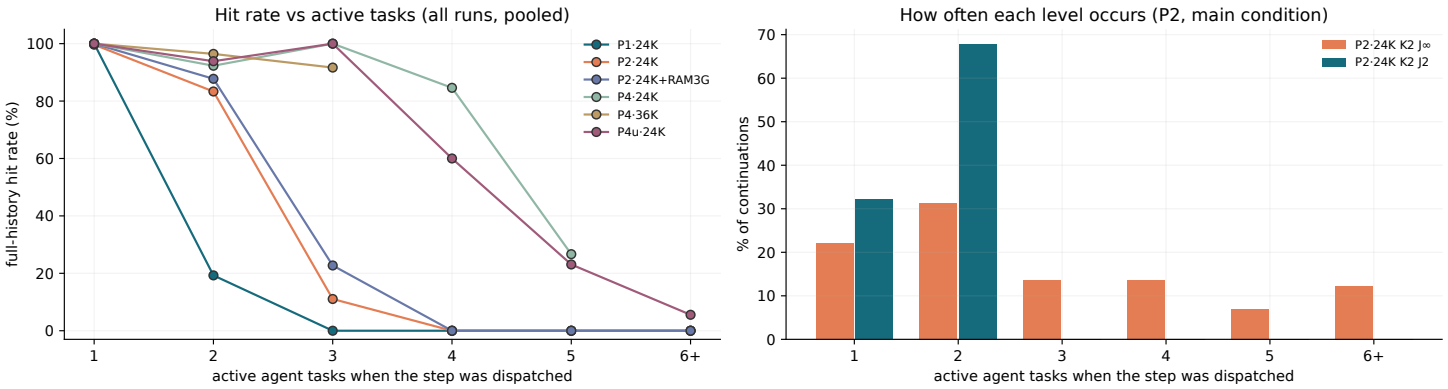
Eval workload (test split), arrivals poisson:1, tool delay lognormal:2:1. P = slots, u = unified KV, K = request concurrency, J = active-task limit. Each repeat is a different seeded mix of 8 tasks with its own arrival pattern (R1 fewest steps, R2 burstiest arrivals).

F4 End-to-end FIFO baselines by server configuration and KV budget



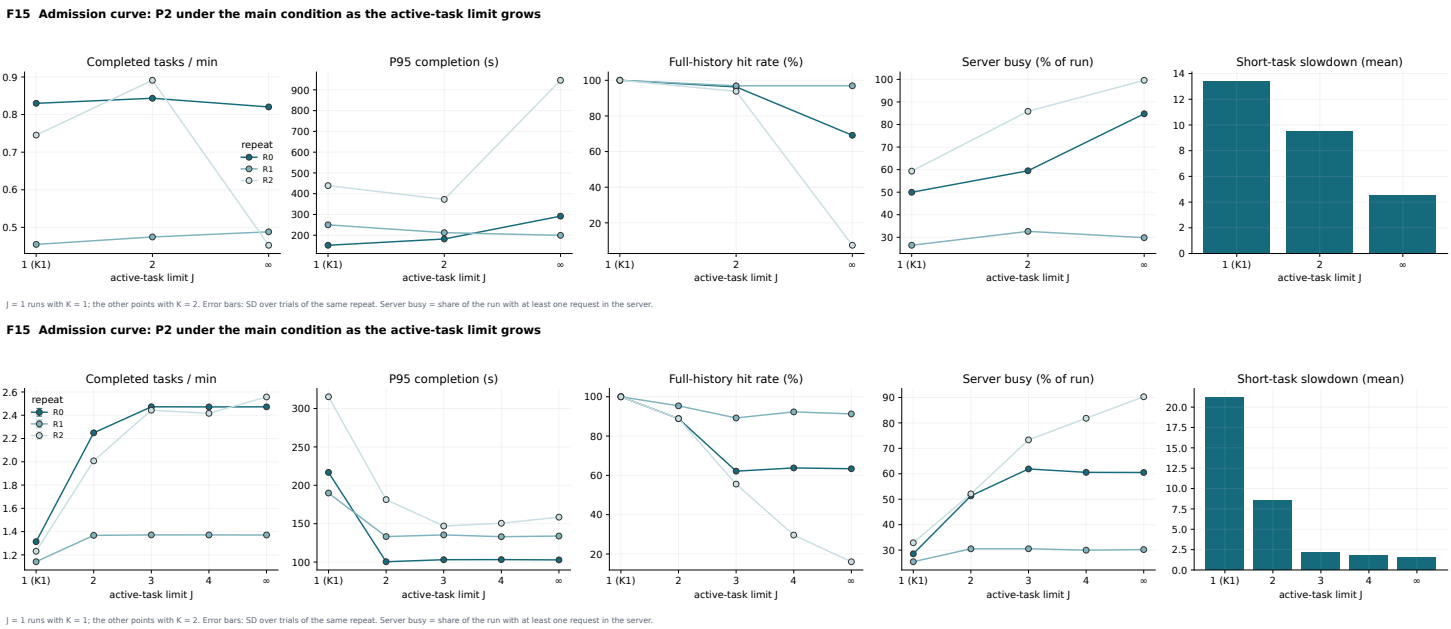
Eval workload (test split), arrivals scaled:3.5, tool delay lognormal:2:1. P = slots, u = unified KV, K = request concurrency, J = active-task limit. Each repeat is a different seeded mix of 8 tasks with its own arrival pattern (R1 fewest steps, R2 burstiest arrivals).

Figure A2: Baselines on the M4 (top) and the T4 (bottom), per repeat, relative to FIFO on the same tasks.



F12 Eviction cliff: histories survive only while active tasks fit the slots
Hit rate vs active tasks (all runs, pooled)
full-history hit rate (%)
active agent tasks when the step was dispatched
How often each level occurs (P2, main condition)
% of continuations
active agent tasks when the step was dispatched
Active = admitted and not finished (tasks waiting for admission hold no KV). Pooled over K, J, delays and arrivals, so load is a confounder: descriptive, not controlled. Points need ≥ 10 continuations.

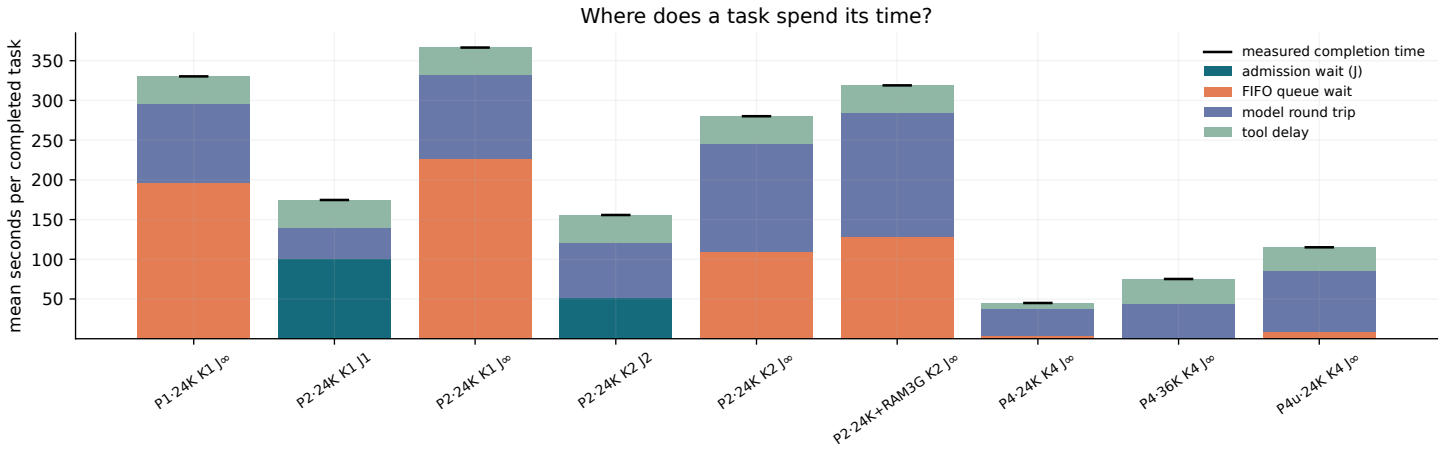
Figure A3: Eviction cliff on the M4 (top) and the T4 (bottom): histories survive only while active tasks fit the slots.



F15 Admission curve: P2 under the main condition as the active-task limit grows
Completed tasks / min
P95 completion (s)
Full-history hit rate (%)
Server busy (% of run)
Short-task slowdown (mean)
active-task limit J
J = 1 runs with K = 1; the other points with K = 2. Error bars: SD over trials of the same repeat. Server busy = share of the run with at least one request in the server.

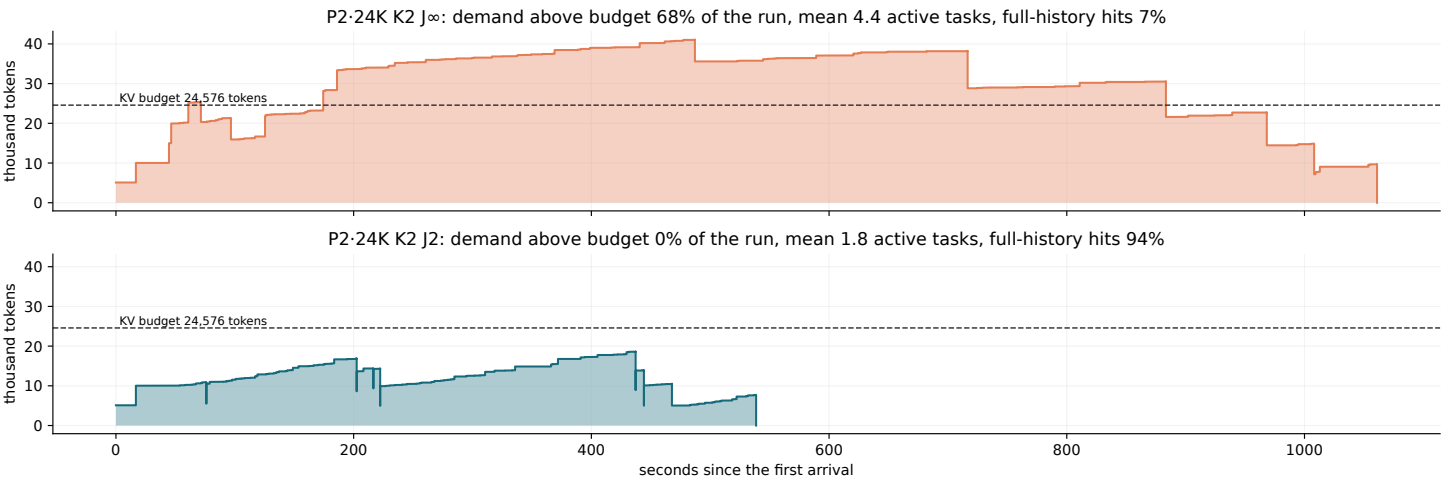
Figure A4: Admission curves on the M4 (top) and the T4 (bottom). On the M4 throughput peaks at $J = 2$; on the T4 it keeps rising with J as the server stays busier.

F5 Completion-time breakdown



Model round trip includes waiting inside llama-server when K > 1 (shared batch).

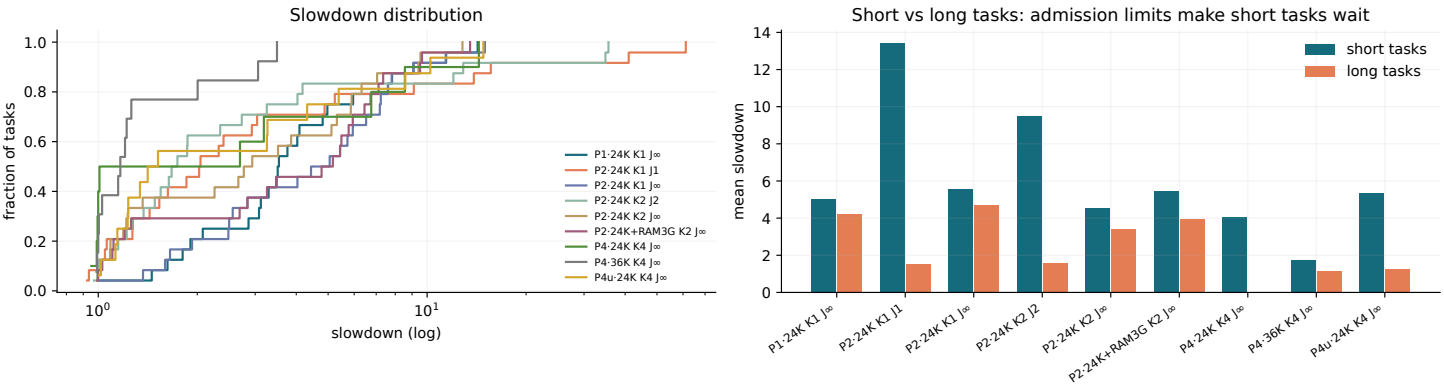
F14 Memory pressure in tokens: KV the active tasks need to keep their histories (R2)



Demand = sum over admitted, unfinished tasks of prompt (+ generated tokens once the step is done). The pool is preallocated (F7), so demand above the budget means histories are evicted.

Figure A5: M4 time per completed task, and KV demand of the admitted tasks against the 24K budget.

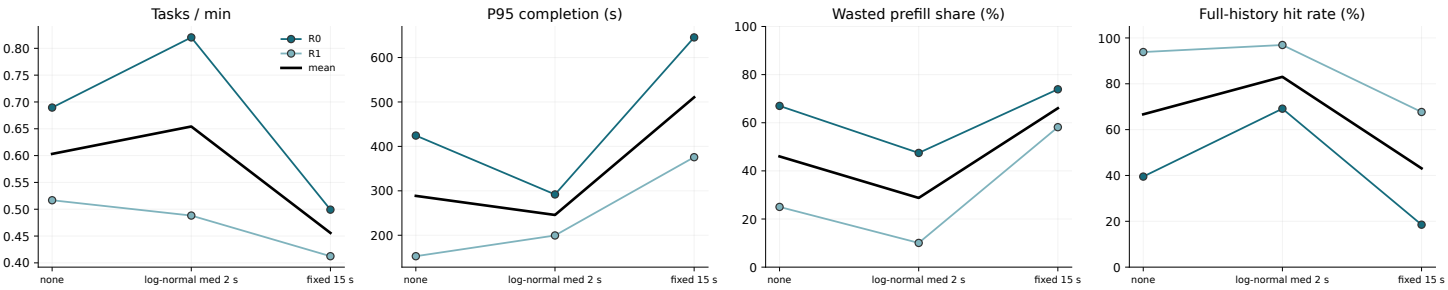
F8 Fairness: slowdown relative to running alone



Slowdown = completion time / (isolated compute time from J=1,K=1 runs + own tool delays). Long = above-median generated tokens.

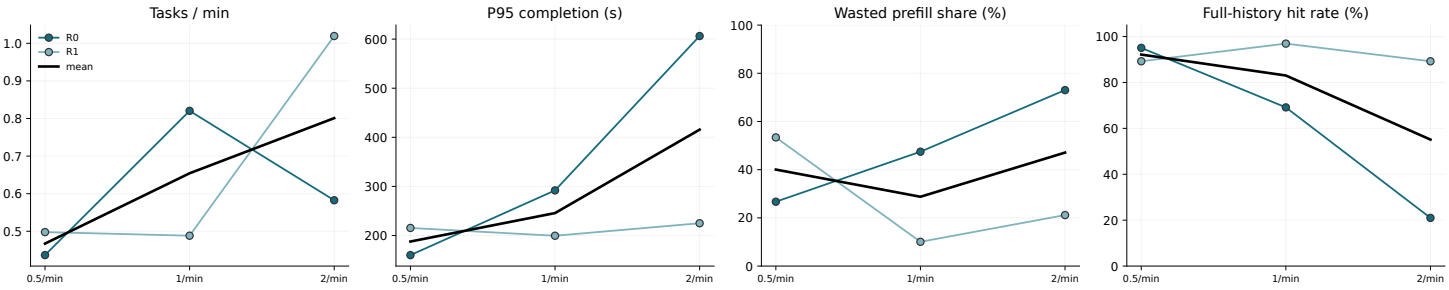
Figure A6: M4 slowdown of short and long tasks.

F10 Effect of tool idle time (P2 K2)



Same arrivals and tasks; only the delay between steps changes. Thin lines: individual repeats (paired task mixes); thick line: mean.

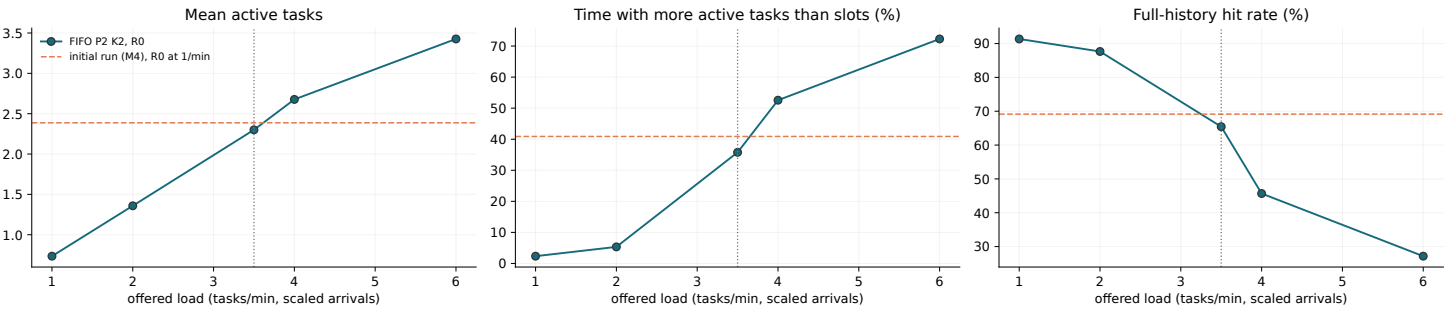
F11 Effect of arrival rate (P2 K2)



Poisson arrivals; tool delay log-normal median 2 s. Thin lines: individual repeats (paired task mixes); thick line: mean.

Figure A7: M4 tool-delay and arrival-rate sweeps ($P = 2$, $K = 2$), compared on the repeats that every setting ran.

F16 Arrival-rate calibration on this machine (pilot runs, not part of the sweep)



Dotted line: the chosen main rate (scaled:3.5). Pilot runs are in runs/aws-gpu-pilot.

Figure A8: T4 arrival-rate calibration: 3.5 tasks/min matches the M4's FIFO contention on R0.

E Timeline

Done

- October 16. We wrote a design memo that treats queue priority and cache retention as one loop. The score and the retention formula in that memo were a plan, and they are not in the code.
- October 16 to 26. We built the FIFO, captured the live Retail tasks, made the append-only replay, and ran the micro benchmark, the cache probe, the restore check, and the M4 eval and dev sweeps.
- October 26. Figures and the generated numeric report.
- October 28 to November 3. M5 noise trials, T4 arrival-rate calibration, and the T4 eval and dev sweeps with three trials of every run.
- November 4. This writeup.

Phase 2

- November 11. Add slot pinning on the current runner, and on the dev split check that a second slot is actually used at $K = 1$.
- November 18. Add a continuation-first queue with aging and cost-based admission for new tasks, and compare it on dev with the best fixed cap of each machine.
- November 25. While a tool runs, choose pin, SSD save, or evict from the costs measured here, and log the decision time and the save time.
- December 2. Run the eval comparison on the M4 and the T4: FIFO, the best fixed baseline, the host cache, queue only, cache only, and both. Include 0.5 arrivals per minute and zero tool delay, and report short-task and long-task slowdown.
- December 8. Freeze the commit, rerun any number the report states, and draft the 4 to 6 page final report.
- December 10. Submit the scheduler, the runner, the raw logs, the analysis scripts, and the final report.